CORRIGÉ · PROBLÈME · ANNALES SPÉ

Chiffrement par flot de booléens

Énoncé

$X(\varphi,\psi) = (\varphi \vee \psi) \wedge (\neg\varphi \vee \neg\psi)$; règles de déduction naturelle en annexe.

1. Arbres de preuve de $X(\varphi,\varphi) \vdash \bot$ et $\bot \vdash X(\varphi,\varphi)$.
2. Arbres de preuve de $X(\bot,\psi) \vdash \psi$ et $\psi \vdash X(\bot,\psi)$.
3. $\oplus(x,y)$ désigne l'évaluation de $X(x,y)$. Justifier que $(x,y) \mapsto \oplus(\oplus(x,x),y)$ et $(x,y) \mapsto y$ sont égales.

LFSR de longueur m, germe $(b_i)_{0\leqslant i<m}$, connexion $C \subseteq [\![0, m-1]\!]$: $b_{n+m} = \bigoplus_{i\in C} b_{n+i}$ ($\oplus$ est associative et commutative ; somme vide $= F$). Germe : `bool array` ; connexion : `int list`.

1. Écrire `lfsr (g:bool array) (c:int list) : bool stream`.
2. Avec `type formula = Var of int | Not of formula | And of formula * formula | Or of formula * formula`, écrire `logical_map (phi:formula) (w:bool stream array) : bool stream` : le flot des évaluations $\varphi(b_n^{(1)},\dots,b_n^{(k)})$ (φ ne dépend que de $x_1,\dots,x_k$, `w` de longueur k).
3. Alice et Bob partagent k LFSR et une formule ψ_0. Alice envoie publiquement $X(\psi_0(b_n^{(1)},\dots,b_n^{(k)}), c_n)$. Bob peut-il retrouver (c_n) ?

Décoder l'énoncé

Ce que l'énoncé te donne.

- $X(\varphi,\psi)$ est le « ou exclusif » de φ et ψ.

Ce qu'on te demande. Prouver en déduction naturelle les propriétés du ou exclusif, puis s'en servir pour chiffrer et déchiffrer avec des LFSR.

Corrigé

Question 24

Étape 1. Q24. $X(\{\}varphi, \{\}varphi)\{\}vdash\{\}bot$

Notons $\Delta = \{X(\varphi, \varphi)\}$ et $X = (\varphi \vee \varphi) \wedge (\neg\varphi \vee \neg\varphi)$.

Arbre 1 ($\Delta \vdash \varphi$) : par $\wedge_e$ on obtient $\varphi \vee \varphi$, puis par $\vee_e$ les deux cas donnent φ par l'axiome.

$$\dfrac{\dfrac{\dfrac{}{\Delta \vdash X}\,(\text{ax})}{\Delta \vdash \varphi \vee \varphi}\,(\wedge_e) \qquad \dfrac{}{\Delta, \varphi \vdash \varphi}\,(\text{ax}) \qquad \dfrac{}{\Delta, \varphi \vdash \varphi}\,(\text{ax})}{\Delta \vdash \varphi}\,(\vee_e)$$

Arbre 2 ($\Delta \vdash \neg\varphi$) : le même, avec la seconde composante.

$$\dfrac{\dfrac{\dfrac{}{\Delta \vdash X}\,(\text{ax})}{\Delta \vdash \neg\varphi \vee \neg\varphi}\,(\wedge_e) \qquad \dfrac{}{\Delta, \neg\varphi \vdash \neg\varphi}\,(\text{ax}) \qquad \dfrac{}{\Delta, \neg\varphi \vdash \neg\varphi}\,(\text{ax})}{\Delta \vdash \neg\varphi}\,(\vee_e)$$

Conclusion par $\neg_e$:

$$\dfrac{\dfrac{\vdots}{\Delta \vdash \varphi}\,(\text{arbre 1}) \qquad \dfrac{\vdots}{\Delta \vdash \neg\varphi}\,(\text{arbre 2})}{X(\varphi, \varphi) \vdash \bot}\,(\neg_e)$$

Réciproque : de $\bot$ on tire n'importe quoi :

$$\dfrac{\dfrac{}{\bot \vdash \bot}\,(\text{ax})}{\bot \vdash X(\varphi, \varphi)}\,(\bot_e)$$

Question 25

Étape 1. Q25. $X(\{\}bot, \{\}psi)\{\}vdash\{\}psi$ et réciproque

Notons $\Delta = \{X(\bot, \psi)\}$, avec $X(\bot, \psi) = (\bot \vee \psi) \wedge (\neg\bot \vee \neg\psi)$.

$$\dfrac{\dfrac{\dfrac{}{\Delta \vdash X(\bot, \psi)}\,(\text{ax})}{\Delta \vdash \bot \vee \psi}\,(\wedge_e) \qquad \dfrac{\dfrac{}{\Delta, \bot \vdash \bot}\,(\text{ax})}{\Delta, \bot \vdash \psi}\,(\bot_e) \qquad \dfrac{}{\Delta, \psi \vdash \psi}\,(\text{ax})}{X(\bot, \psi) \vdash \psi}\,(\vee_e)$$

Réciproque $\psi \vdash X(\bot, \psi)$: on construit les deux composantes et on les assemble par $\wedge_i$. Pour la seconde, $\neg\bot$ s'obtient par $\neg_i$ (sous l'hypothèse $\bot$, on a $\bot$ par l'axiome).

$$\dfrac{\dfrac{\dfrac{}{\psi \vdash \psi}\,(\text{ax})}{\psi \vdash \bot \vee \psi}\,(\vee_i) \qquad \dfrac{\dfrac{\dfrac{}{\psi, \bot \vdash \bot}\,(\text{ax})}{\psi \vdash \neg\bot}\,(\neg_i)}{\psi \vdash \neg\bot \vee \neg\psi}\,(\vee_i)}{\psi \vdash X(\bot, \psi)}\,(\wedge_i)$$

Question 26

Étape 1. Q26. $\{\}oplus(\{\}oplus(x,x),y) = y$

$\oplus(x,y)$ vaut V si et seulement si exactement un de x, y vaut V : $(x \vee y)$ exige au moins un V, et $(\neg x \vee \neg y)$ au moins un F.

- $\oplus(x,x) = F$ pour tout x (au moins un V et au moins un F parmi x, x est impossible) : c'est la version sémantique de Q24.
- $\oplus(F,y) = y$: si $y = V$, exactement un vaut V ; si $y = F$, aucun. C'est la version sémantique de Q25.

Donc pour tous x, y : $\oplus(\oplus(x,x),y) = \oplus(F,y) = y$. Les deux fonctions sont égales.

x	y	$\oplus(x,x)$	$\oplus(\oplus(x,x),y)$
V	V	F	V
V	F	F	F
F	V	F	V
F	F	F	F

Question 27

Étape 1. Q27. Le LFSR

```
let lfsr (g : bool array) (c : int list) : bool stream =
  let m = Array.length g in
  (* w contient la fenêtre (b_n, ..., b_{n+m-1}) *)
  let rec from (w : bool array) : bool stream =
    Cons (w.(0), fun () ->
      let next = List.fold_left (fun acc i -> acc <> w.(i)) false c in   (* b_{n+m} *)
      from (Array.init m (fun i -> if i < m - 1 then w.(i + 1) else next)))
  in
  if m = 0 then Nil else from (Array.copy g)
```

Sur les booléens, `a <> b` vaut `true` si et seulement si exactement un des deux est vrai : c'est $\oplus$. Le `fold_left` part de `false` (F, somme vide) et ajoute les b_{n+i} pour $i \in C$. On copie le germe pour ne pas dépendre de modifications ultérieures du tableau passé en argument.

Question 28

Étape 1. Q28. logical_map

```
let rec eval (phi : formula) (b : bool array) : bool =
  match phi with
  | Var j -> b.(j - 1)                (* x_j est la j-ième valeur, indice j-1 *)
  | Not p -> not (eval p b)
  | And (p, q) -> eval p b && eval q b
  | Or (p, q) -> eval p b || eval q b
```

```
let logical_map (phi : formula) (w : bool stream array) : bool stream =
  map (eval phi) (zip w)
```

`zip w` fournit, pour chaque n, le tableau $[b_n^{(1)}, \dots, b_n^{(k)}]$; `map` lui applique l'évaluation de φ.

Question 29

Étape 1. Q29. Bob déchiffre

Oui, toujours. Bob connaît les germes, les connexions et ψ_0 : il recalcule exactement les mêmes flots $b^{(1)}, \dots, b^{(k)}$ (avec `lfsr`) et le même flot de clé $k_n = \psi_0(b_n^{(1)}, \dots, b_n^{(k)})$ (avec `logical_map`). Il reçoit $e_n = \oplus(k_n, c_n)$ et calcule

$$\oplus(k_n, e_n) = \oplus(k_n, \oplus(k_n, c_n)) = \oplus(\,\oplus\,(k_n, k_n), c_n) = c_n,$$

par associativité de $\oplus$ puis Q26. Un espion qui ne connaît pas la clé ne voit que e_n.

Le piège

En Q25, oublier que $\neg\bot$ se démontre (par $\neg_i$) : ce n'est pas une hypothèse disponible.

La méthode

Chiffrement par flot : message $\oplus$ clé ; on déchiffre en refaisant $\oplus$ clé, car $k \oplus k = F$ et $F \oplus c = c$.

Rappels de cours utilisés

- **Déduction naturelle (règles utilisées).** (ax) $\Delta, A \vdash A$; $(\wedge_e)$ de $A \wedge B$ on tire A (ou B) ; $(\wedge_i)$ de A et B on tire $A \wedge B$; $(\vee_i)$ de A on tire $A \vee B$; $(\vee_e)$ de $A \vee B$, $\Delta, A \vdash C$, $\Delta, B \vdash C$ on tire C ; $(\neg_i)$ de $\Delta, A \vdash \bot$ on tire $\neg A$; $(\neg_e)$ de A et $\neg A$ on tire $\bot$; $(\bot_e)$ de $\bot$ on tire tout.